

Variables and Combining Commands

In *professional settings*, UPDATING programs for **everyone in a company** is a big decision. MEANING, it's something they'd like to keep track of.

Let's make a script called `update_log`, that **updates and logs the update** for each user.

In the **first line**, let's go to the *home folder*.

```
> cd ~
```

Let's use our first variable.

```
> echo "$USER is updating their programs"
```

`USER` is a **built in variable**, we *CALL* the variable, meaning getting what it stores via the \$.

- Which *MUST* be in between `""` → `"$USER (you can have other stuff in here too)"`
-

Now, let's make **our own variable**.

```
> LOG="${USER}'s_update.log"
```

`LOG` is the *name of our variable*, wrapped in `""`

Notice, how we have a `{}` around the USER variable. This is because, we have *other words* beside our variable.

- Without it; it'll look for a variable called `USER's_update.log`

Now, we've made a *variable log* that is **different for each user**.

- This *doesn't* actually MAKE the file **just the name**, so we *have to do that*.

```
> touch "$LOG" 2>/dev/null
```

ACTUALLY Makes the file, that has the SAME NAME as the variable, REMEMBER; `2>/dev/null` just *deletes any error if the file exists already*.

Ok, now let's **actually** update.

> `sudo apt update ; sudo apt upgrade -y >> "$LOG"` The `;` signifies, WAIT for the command on the left to **finish** AND THEN run the next one. Then the `>>` puts the update information into the log file.

- This is because, upgrade **NEEDS** the files that update gets from the internet to actually run the upgrades.

Lastly, let's add a little command to tell you it's done, AND add the date.

> `echo "Done!" & echo "On $(date) $USER updated." >> "$LOG"` The first command, just spits out to the terminal "it is done".

- More importantly, the `&` allows both commands to be run at the same time. Since the 2nd command does NOT care about the first one to finish.

In the second command, the `()` in the `$(date)` allows us to have "date" run the command somewhere else, and give us back the output. Then the `$USER` give us the specific username, and the `>>` adds it to the log file.

Your script, SHOULD look like this;

```
#!/usr/bin/env bash
cd ~

echo "$USER is updating"

LOG="${USER}_update.log"

touch "$LOG" 2>/dev/null

sudo apt update ; sudo apt upgrade -y >> "$LOG"

echo "Done!" & echo "On $(date), $USER updated their programs" >> "$LOG"
```

That's it for this PDF, now you have the *fundamentals* for making a surprisingly good amount of scripts. Next lab will cover IF statements and reading user input.